FLUTTER MASTERCLASS DERS SUNUMU

Flutter Tasarım, Temalar ve Etkileşim Rehberi

Tasarım Sistemleri • Tipografi • GPU Shaders • State Management • Gestures • Forms • Keyboard & Focus (22 Bölüm — Tam Kılavuz)

KISIM I & II	KISIM III	KISIM IV & V
Görsel Sistemler & Temalar	**Etkileşim & Durum Yönetimi**	**Formlar & Gelişmiş Odak**
• ThemeData & ColorScheme Mimarisi • Material Design 3 & Cupertino iOS • TextTheme Tipografi Ölçeği • GPU GLSL Fragment Shaders	• Stateful/Stateless Widget Mantığı • Pointer Events & Gesture Arena • Drag & Drop (Uygulama İçi & Dışı) • InkWell & Dismissible Listeler	• Form, TextFormField & GlobalKey • TextEditingController & Listeners • Keyboard Shortcuts & Intent/Action • FocusNode & FocusTraversalPolicy

Kısım I: Tasarım Sistemleri • Bölüm 1

1. Renkleri ve Stilleri Paylaşmak İçin Tema Kullanma

Tema Mimarisi & Hiyerarşi

ThemeData Yapılandırması

• Flutter, stil öncelik sırasını 3 katmanda uygular: 1. Widget özel stili 2. Yakın üst tema (Override) 3. Ana uygulama teması.

• ColorScheme.fromSeed() ile tek renk seed'inden uyumlu tüm renk paletleri türetilir.

• Theme.of(context) widget ağacında yukarı doğru arama yaparak en yakın temayı yakalar.

• copyWith() metodu var olan üst temayı bozmadan yalnızca belirli özellikleri geçersiz kılar.

Kod Uygulaması

Örnek Dart Kod Bloğu

```
// Uygulama Seviyesi Tema MaterialApp( theme: ThemeData(
colorScheme: ColorScheme.fromSeed( seedColor: Colors.deepPurple,
brightness: Brightness.dark, ), textTheme: TextTheme( displayLarge:
TextStyle(fontSize: 32, fontWeight: FontWeight.bold), ), ), ); //
Lokal Temayı Genişletme (copyWith) Theme( data:
Theme.of(context).copyWith( colorScheme:
ColorScheme.fromSeed(seedColor: Colors.pink), ), child:
FloatingActionButton(onPressed: () {}), );
```

Kısım I: Tasarım Sistemleri • Bölüm 2 & 3

2 & 3. Material Design 3 ve Cupertino (iOS) Tasarım Sistemleri

Google Material 3

Material Design 3 (Android & Web)

• Google'ın açık kaynaklı yeni nesil tasarım dilidir (Material You).

• Dinamik renk geçişleri, yumuşatılmış bileşen köşeleri ve erişilebilirlik standartları sunar.

• ElevatedButton, FloatingActionButton, NavigationRail gibi hazır zengin bileşen kataloğuna sahiptir.

Apple Cupertino

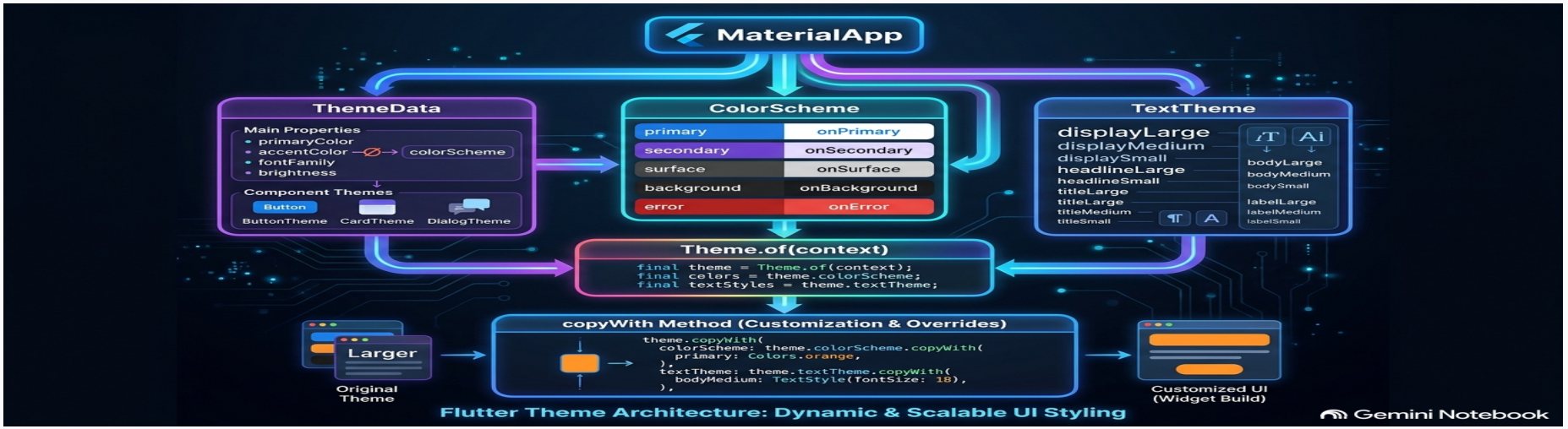
Apple Cupertino Ekosistemi (iOS)

• iOS platformunun doğal görünüm, his ve etkileşim dilini yansıtır.

• Yuvarlatılmış köşe yarıçapları, buzlu cam (blur) efektleri ve yumuşak kaydırma fiziği içerir.

• cupertino_ui paketinden erişilir: CupertinoNavigationBar, CupertinoSwitch, CupertinoSheetRoute.

Şematik Görsel: Flutter Tema Mimarisi Hiyerarşisi



Kısım II: Tipografi ve Grafik Efektleri • Bölüm 4, 5, 6

4, 5 & 6. TextTheme Tipografi Ölçeği ve Font Yönetimi

Tipografi & Değişken Fontlar

TextTheme (15 Düzeyli Tipografik Ölçek)

• Metinler 5 ana kategoride toplanır: Display, Headline, Title, Body, Label.

• Her kategori Large, Medium, Small alt seviyelerine ayrılır (toplam 15 stil).

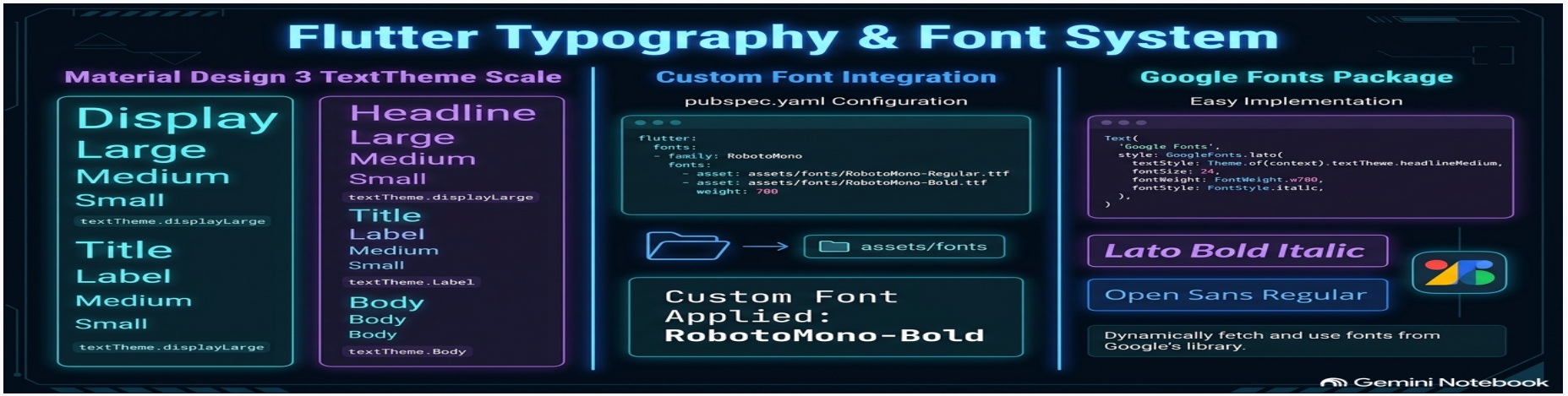
• Statik fontlar tek ağırlık taşırken, Variable Fonts (Değişken Fontlar) font boyutu, kalınlık ve eğikliği tek dosyada programatik değiştirmeyi sağlar.

Pubspec & Paket Tanımları

pubspec.yaml ile Özel Font Tanımlama

```
# pubspec.yaml Font Yapılandırması flutter: fonts: - family: CustomRaleway fonts: - asset: fonts/Raleway-Regular.ttf - asset: fonts/Raleway-Italic.ttf style: italic - asset: fonts/Raleway-Bold.ttf weight: 700 # Paket İçi Font Kullanımı TextStyle( fontFamily: 'MyFont', package: 'my_package_name', )
```

Tipografi Ölçeği & Değişken Font Değişim Şeması



Kısım II: Tipografi ve Grafik Efektleri • Bölüm 7

7. GPU Tabanlı Parça Gölgelendiricileri (Fragment Shaders)

GPU & GLSL Shader Mimarisi

Fragment Shader İlkeleri

• Flutter hem Skia hem de Impeller grafik motorunda GPU piksel bazlı GLSL (OpenGL Shading Language) çalıştırmayı destekler.

• FragmentProgram.fromAsset() ile .frag dosyaları çalışma zamanında yüklenir.

• setFloat() ile koordinat, zaman ve piksel verileri Dart'tan GPU'ya iletilir.

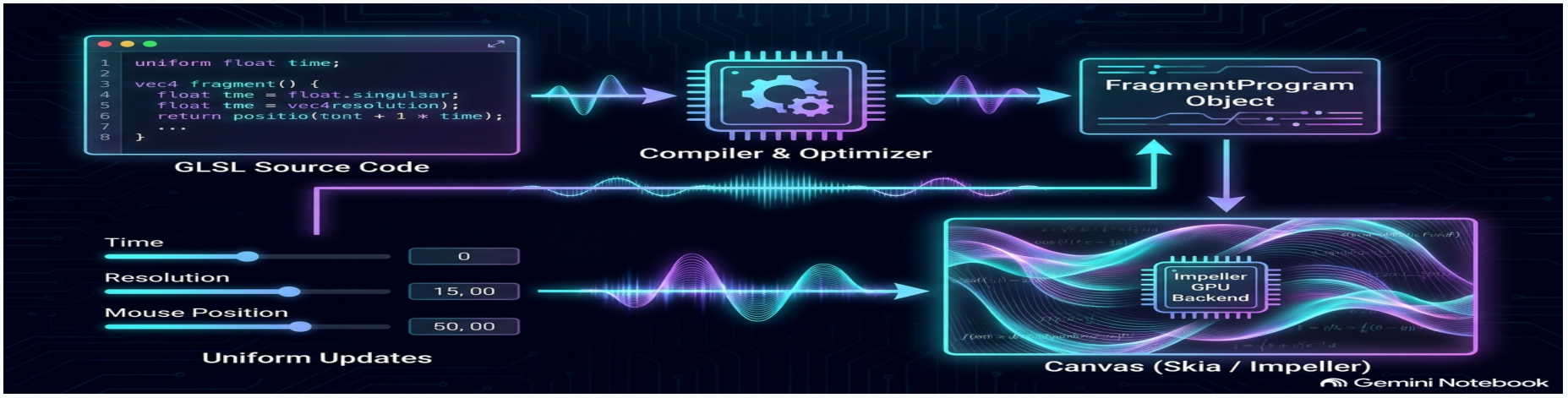
• Canvas.drawRect veya ImageFilter.shader ile piksel efektleri çizilir.

Dart & GLSL Entegrasyonu

GLSL & Dart Entegrasyon Örneği

```
// GLSL Kodu (my_shader.frag) #version 460 core #include out vec4 fragColor; uniform vec2 uSize; void main() { vec2 uv = FlutterFragCoord().xy / uSize; fragColor = vec4(uv.x, uv.y, 0.8, 1.0); } // Dart Kodu final program = await FragmentProgram.fromAsset('shaders/my_shader.frag'); final shader = program.fragmentShader(); shader.setFloat(0, size.width); shader.setFloat(1, size.height); canvas.drawRect(rect, Paint()..shader = shader);
```

GPU Fragment Shader İşleme Hattı (Impeller & Skia)



Kısım III: Etkileşim ve Durum Yönetimi • Bölüm 8

8. Flutter Uygulamanıza Etkileşim Ekleme & Durum Yönetimi

Widget & State Sınıfları

Stateless vs Stateful Widget Mimarisi

• StatelessWidget: Hiçbir zaman değişmeyen, sabit arayüz öğeleridir (örn. Text, Icon).

• StatefulWidget: Kullanıcı etkileşimi veya veri akışıyla dinamik değişen widget'lardır.

• Durum verileri, widget'tan ayrı olan State sınıfında saklanır.

• setState() çağrısı framework'e durumun değiştiğini bildirir ve build() metodunu yeniden tetikler.

3 Temel Yaklaşım

Durum Yönetimi Stratejileri (Tapbox Örnekleri)

1. Widget Kendi Durumunu Yönetir (TapboxA): İç etkileşimler için durum tamamen widget içinde kalır.

2. Üst Widget Durumu Yönetir (TapboxB): Veri üst bileşene aittir, onChanged callback'i ile durum yukarı iletilir.

3. Karma Yaklaşım (TapboxC): Görsel efektler (vurgu/border) widget içinde, veri durumu üst öğede yönetilir.

StatefulWidget & setState() Kod Örneği

```
class FavoriteWidget extends StatefulWidget { const FavoriteWidget({super.key}); @override State createState() => _FavoriteWidgetState(); }
class _FavoriteWidgetState extends State { bool _isFavorited = true; int _favoriteCount = 41; void _toggleFavorite() { setState(() { if
(_isFavorited) { _favoriteCount -= 1; _isFavorited = false; } else { _favoriteCount += 1; _isFavorited = true; } }); } }
```

Kısım III: Etkileşim ve Durum Yönetimi • Bölüm 9, 10, 13, 14

9, 10, 13 & 14. İşaretçi Olayları, Jest Arenası & Efektler

İşaretçiler & Gesture Arena

İşaretçi (Pointer) & Jest Katmanları

• Ham İşaretçi Olayları: PointerDownEvent, PointerMoveEvent, PointerUpEvent, PointerCancelEvent.

• İsabet Testi (Hit Test): Dokunulan konumdaki widget belirlenir ve olaylar widget ağacında yukarı doğru kabarcıklanır (bubble up).

• Gesture Arena (Müzakere): Çakışan jestler arasında (örn. dikey kaydırma vs uzun basma) tek bir kazanan jest belirlenir.

InkWell & Dismissible

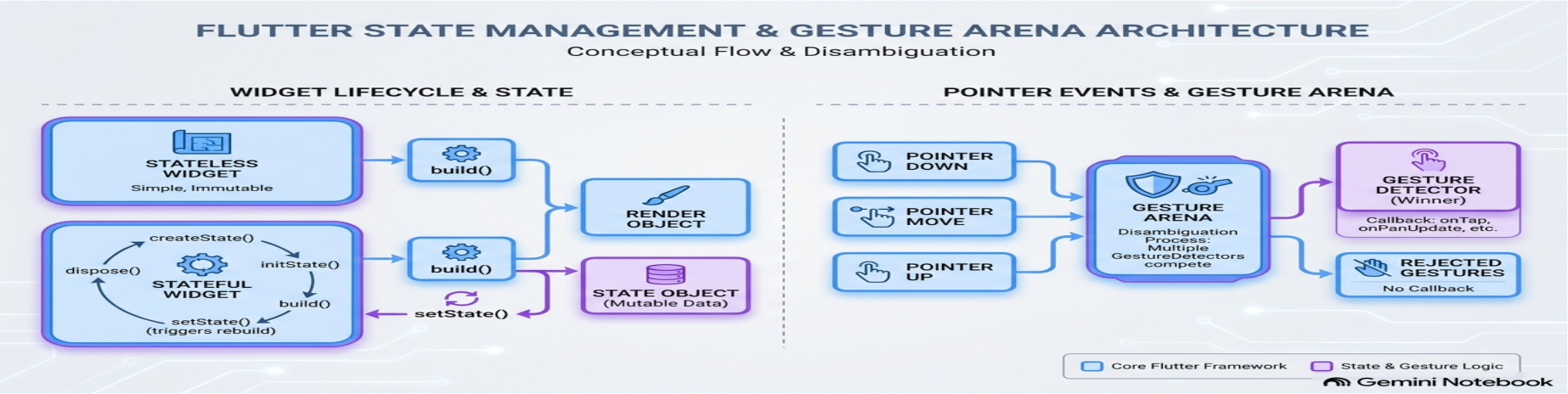
Etkileşim Bileşenleri & Efektler

• GestureDetector: Dokunma (onTap), çift dokunma (onDoubleTap), uzun basma ve sürükleme olaylarını yakalar.

• InkWell: Material tasarım standartlarında dokunma dalgalanması (ripple effect) animasyonu üretir.

• Dismissible: Öğeleri sola/sağa kaydırarak listeden silme mantığı ve arka plan görseli (background) sağlar.

Şematik Görsel: Flutter State Management & Gesture Arena İşleyişi



Kısım III: Etkileşim ve Durum Yönetimi • Bölüm 11 & 12

11 & 12. Uygulama İçi ve Uygulamalar Arası Sürükle-Bırak

Sürükle-Bırak Mekanizması

LongPressDraggable ve DragTarget (Uygulama İçi)

• LongPressDraggable: Kullanıcı ögeye uzun bastığında taşınabilir feedback widget'ını başlatır.

• dragAnchorStrategy: pointerDragAnchorStrategy ile görsel parmağın tam altında konumlanır.

• data parametresi taşınacak nesne verisini barındırır.

• DragTarget: Sürüklenen nesneyi kabul eder; onAcceptWithDetails tetiklendiğinde hedef güncellenir.

Kod & Paket Kullanımı

Uygulamalar Arası Sürükleme (super_drag_and_drop)

• İşletim sistemi seviyesinde diğer uygulamalara veri aktarımı için super_drag_and_drop paketi kullanılır.

• Masaüstü, mobil ve web platformlarında senkron veri eşleşmesi sağlar.

```
// LongPressDraggable Örneği LongPressDraggable( data: item, feedback: DraggingListItem(photo: item.image), child: MenuListItem(name: item.name), ); // DragTarget Örneği DragTarget( onAcceptWithDetails: (details) => cart.add(details.data), builder: (context, candidateData, rejectedData) => CartWidget(), );
```

Kısım IV: Formlar ve Metin Girişi • Bölüm 15, 16, 17

15, 16 & 17. TextField, TextFormField ve TextEditingController

Metin Alanları & Denetleyiciler

TextField & TextFormField Stillendirme

• TextField temel metin giriş bileşenidir; InputDecoration ile kenarlık (border), ipucu (hintText) ve etiket (labelText) tanımlanır.

• TextFormField, TextField'ı sarmalar ve Form doğrulama mekanizmalarına bağlar.

• TextEditingController: Metin alanındaki değeri controller.text ile okur.

• Kaynak Temizliği: Bellek sızıntılarını önlemek için State sınıfının dispose() metodunda controller.dispose() çağrılmalıdır.

Değişiklik Dinleme Mantığı

Değişiklikleri Dinleme Yöntemleri

1. onChanged() Callback: Anlık basit değişiklikler için kullanılır.
2. addListener(): Controller seviyesinde dinleme başlatır (initState'te eklenir, dispose'da çıkarılır).

```
// Anlık Dinleme Örneği class _MyFormState extends State { final
_controller = TextEditingController(); @override void initState() {
super.initState(); _controller.addListener(() { print("Metin:
${_controller.text}"); }); } @override void dispose() {
_controller.dispose(); super.dispose(); } }
```

Kısım IV: Formlar ve Metin Girişi • Bölüm 18 & 19

18 & 19. Odak Yönetimi (FocusNode) ve Form Doğrulama

Programatik Odak Kontrolü

Odak Yönetimi (FocusNode)

• autofocus: true: Ekran açılır açılmaz odağı metin alanına yönlendirir.

• FocusNode: Programatik odak kontrolü sağlar; focusNode.requestFocus() ile odak istenen alana taşınır.

• FocusNode nesneleri State içinde oluşturulmalı ve dispose() edilmelidir.

Form Validation & GlobalKey

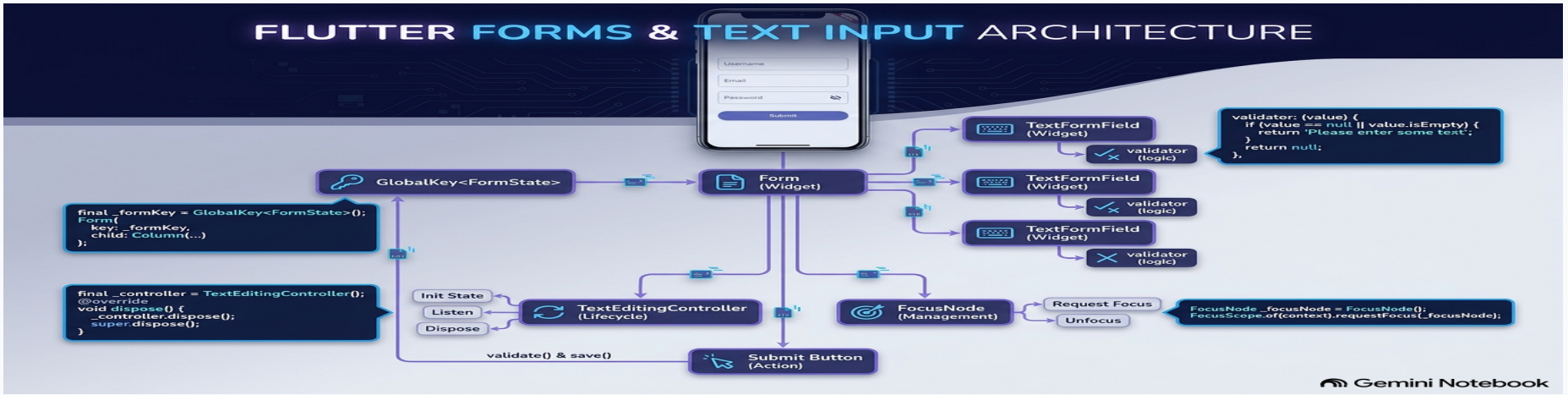
GlobalKey ile Form Doğrulama (Validation)

• final _formKey = GlobalKey() ile Form kontrol edilir.

• TextFormField.validator: Hata durumunda mesaj metni (String), başarılıysa null döndürür.

• _formKey.currentState!.validate() tüm form alanlarını doğrular.

Şematik Görsel: Flutter Form Validation & Focus Tree Yapısı



Kısım V: Bildirimler, Kısayollar ve Odak • Bölüm 20

20. SnackBar ve ScaffoldMessenger Bildirim Sistemi

ScaffoldMessenger Mantığı

ScaffoldMessenger ve SnackBar Mimarisi

• Flutter'da hızlı alt bildirim mesajları göstermek için SnackBar kullanılır.

• ScaffoldMessenger.of(context) widget ağacında mesaj yöneticisini bulur ve showSnackBar() ile ekrana yansıtır.

• Eski Scaffold.of(context).showSnackBar yapısının yerini alan güvenli yeni standarttır.

SnackBarAction Kullanımı

Etkileşimli Eylem (SnackBarAction) Örneği

```
// SnackBar Oluşturma ve Tetikleme final snackBar = SnackBar( content: const Text('Öğe başarıyla silindi'), action: SnackBarAction( label: 'Geri Al (Undo)', onPressed: () { // Silme işlemini geri alma kodu }, ), ); // Ekranda Görüntüleme ScaffoldMessenger.of(context).showSnackBar(snackBar);
```

Kısım V: Bildirimler, Kısayollar ve Odak • Bölüm 21

21. Eylemleri ve Klavye Kısayollarını Kullanma (Actions & Shortcuts)

Klavye Eylem Hiyerarşisi

Intent, Action ve Shortcuts Kavramları

• Shortcuts: Fiziksel tuş kombinasyonlarını (örn. Ctrl+A) soyut niyetlere (Intent) eşler.

• Intent: Kullanıcının yapmak istediği genel amacı temsil eder (örn. SelectAllIntent).

• Actions: Niyeti karşılayan ve gerçek iş mantığını yürüten sınıftır (örn. SelectAllAction).

• Bu ayrım, aynı kısayolun odaktaki bileşene göre farklı eylem çalıştırmasını sağlar.

Kısayol Tanımları & Kod

CallbackShortcuts & ShortcutManager

• Basit kısayol ihtiyaçları için CallbackShortcuts pratik bir alternatiftir.

• LoggingActionDispatcher ile eylem çağrıları merkezi olarak günlüğe yazılabilir.

```
// CallbackShortcuts Örneği CallbackShortcuts( bindings: { const
SingleActivator(LogicalKeyboardKey.arrowUp): () { setState(() =>
count++); }, }, child: Focus(autofocus: true, child:
CounterWidget()), );
```

Kısım V: Bildirimler, Kısayollar ve Odak • Bölüm 22

22. Flutter Klavye Odak Sistemi (Focus, FocusScope & Traversal)

Focus Ağacı & Bileşenler

Odak Sistemi Temel Bileşenleri

• Focus Widget: Bir FocusNode yöneten ve tuş olaylarını (onKeyEvent) dinleyen ana bileşendir.

• FocusScope: Düğümleri gruplayan odak kapsama alanıdır.

• FocusManager: Sistemdeki birincil odağı (primaryFocus) korur.

• FocusableActionDetector: Focus, Shortcuts, Actions ve MouseRegion bileşenlerini tek pakette birleştirir.

Focus Traversal & Best Practices

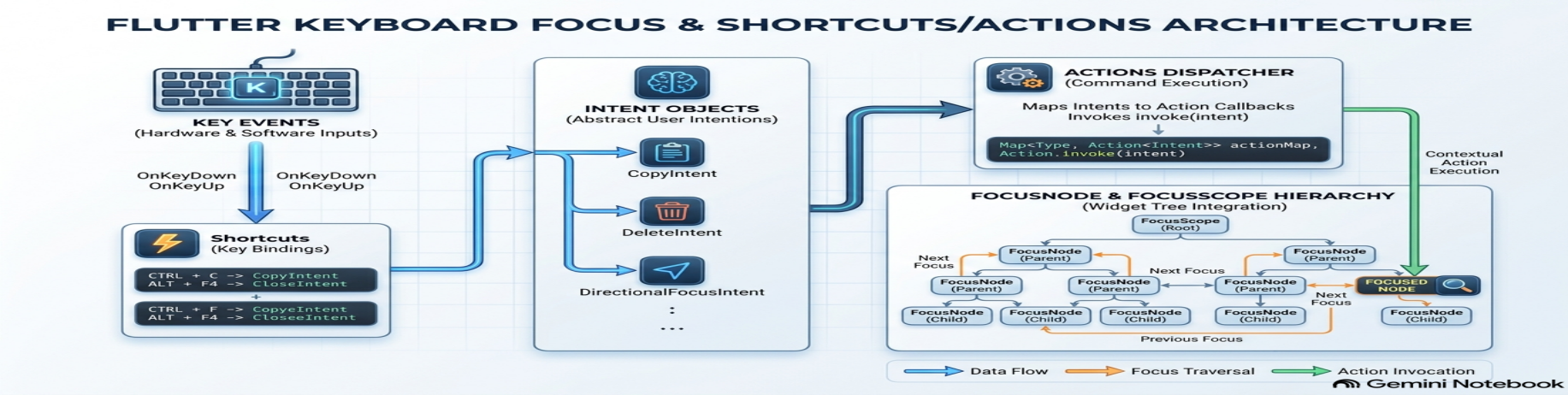
Sekme Gezintisi (Focus Traversal) & Kurallar

• Varsayılan gezinti ReadingOrderTraversalPolicy (okuma sırası) ile çalışır.

• FocusTraversalGroup ve NumericFocusOrder ile sıralama özel olarak belirlenebilir.

• Önemli Kural: FocusNode nesneleri build() içinde oluşturulmamalı, mutlaka StatefulWidget içinde yönetilmelidir.

Şematik Görsel: Flutter Keyboard Focus & Shortcuts/Actions Mimarisi



Özet ve Genel Değerlendirme

Flutter Tasarım, Temalar ve Etkileşim Mimarisi Özeti

Görsel & Grafik Temelleri

Tasarım & Tipografi İlkeleri (Kısım I & II)

1. Merkezi Temalama: ThemeData ve ColorScheme.fromSeed ile tutarlı renk/font paletleri.
2. Çoklu Tasarım Dili: Material Design 3 ve Cupertino (iOS) bileşen uyumluluğu.
3. Tipografi & Fontlar: 15 seviyeli TextTheme ve Değişken Font esnekliği.
4. Gelişmiş Grafikler: Impeller/Skia GPU üzerinde çalışan custom GLSL Fragment Shaders.

Etkileşim & Girdi Temelleri

Etkileşim & Girdi Sistemleri (Kısım III, IV & V)

5. Reaktif Durum: Stateful/Stateless widget ayrımı, setState() ve kapsüllenmiş durum yönetimi.
6. Jest & Sürükleme: Gesture Arena müzakeresi, LongPressDraggable ve Dismissible.
7. Güvenli Formlar: TextEditingController, GlobalKey ve TextFormField doğrulaması.
8. Masaüstü & Web Uyumu: Keyboard Shortcuts, Intent/Action ayrımı ve Focus Traversal politikaları.

Ders Sonu Notu

Tebrikler! Flutter Tasarım ve Etkileşim Rehberi'nin tüm 22 Bölümünü Tamamladınız.

Bu sunum dokümanı, resmi Flutter rehber dokümantasyonu esas alınarak ders anlatım standartlarında hazırlanmıştır.